REINFORCEMENT LEARNING

Unit 5 — Real-World RL Homework Project

Project 7: Optimizing Chemical Reaction Yield

Self-Driving Laboratory Simulator Using Q-Learning

Submitted by	APPIDI SANJANA
Student ID	100523733063
Course	Reinforcement Learning — Unit 5
Project category	RL for Scientific Discovery

Project overview

A simulated laboratory uses tabular Q-learning to select temperature and concentration adjustments that improve reaction yield. Performance is evaluated against a random-action baseline over 700 training episodes, with 35 simulated experiments per episode.

Included in this submission

Problem formulation, Q-learning method, experimental results and figures, interpretation, limitations, responsible-use considerations, and conclusion.

Optimizing Chemical Reaction Yield Using Reinforcement Learning

Project 7: Self-Driving Laboratory Simulator | Unit 5 – Reinforcement Learning

Abstract. This mini-project develops a simulated self-driving laboratory in which a Q-learning agent searches for temperature and concentration settings that produce a high chemical reaction yield. The agent is compared with a random-action baseline. The environment is synthetic and is intended for education; it does not represent a validated chemical process.

1. Problem formulation

Objective: Find experimental settings that maximize simulated reaction yield while learning from repeated experiments.

Environment: A two-dimensional grid of 11 temperature values (30–100 °C) and 11 concentration values (0.1–1.0 relative units). One episode contains 35 sequential experiments. The simulator returns a yield between 0% and 100%, with small measurement noise.

State: Current temperature-grid index and concentration-grid index (121 possible states).

Actions: Nine local moves: increase, decrease or hold temperature and concentration by one grid step.

Reward: Observed reaction yield divided by 100, giving a reward in [0,1]. Higher yield gives a larger reward.

Learning algorithm: Tabular Q-learning with epsilon-greedy exploration. Learning rate α=0.18, discount factor γ=0.92, and epsilon decays from 1.0 to 0.04.

2. Method

At each step, the agent observes its current settings, chooses one of nine adjustments, performs a simulated experiment, receives the yield as reward, and updates its Q-table. During early training it explores many actions; later it increasingly selects actions with high learned Q-values. A random-action agent receives the same environment, episode length and experiment budget as the baseline.

3. Results

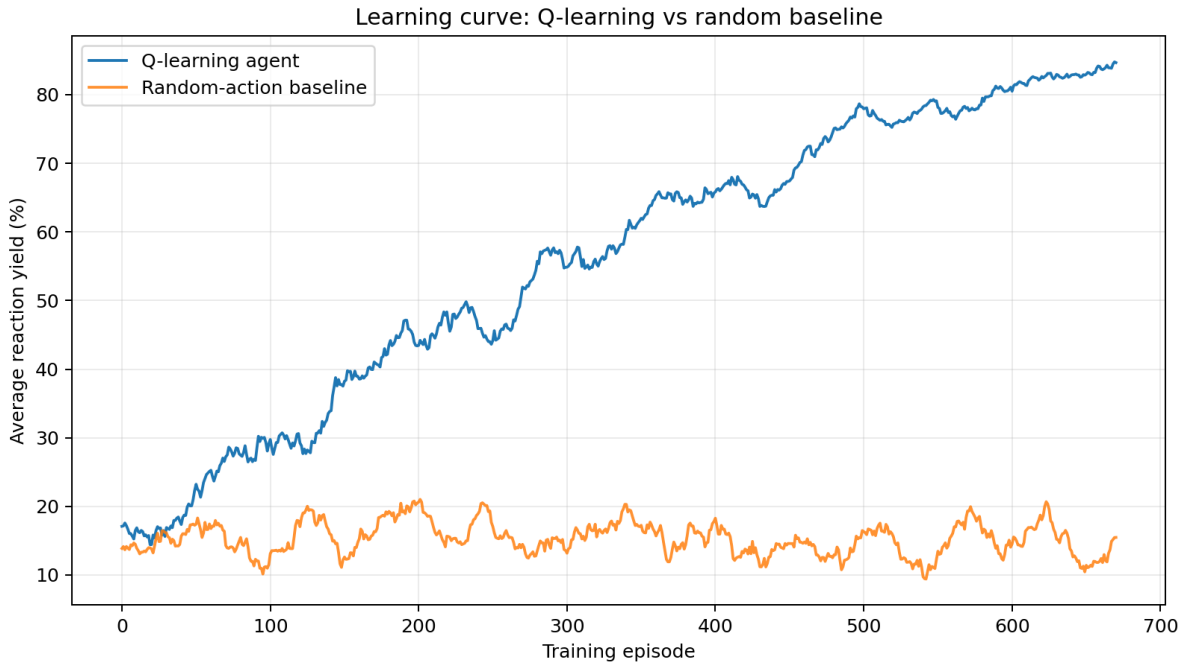


Figure 1. Thirty-episode moving average of mean yield per episode for the learning agent and random-action baseline.

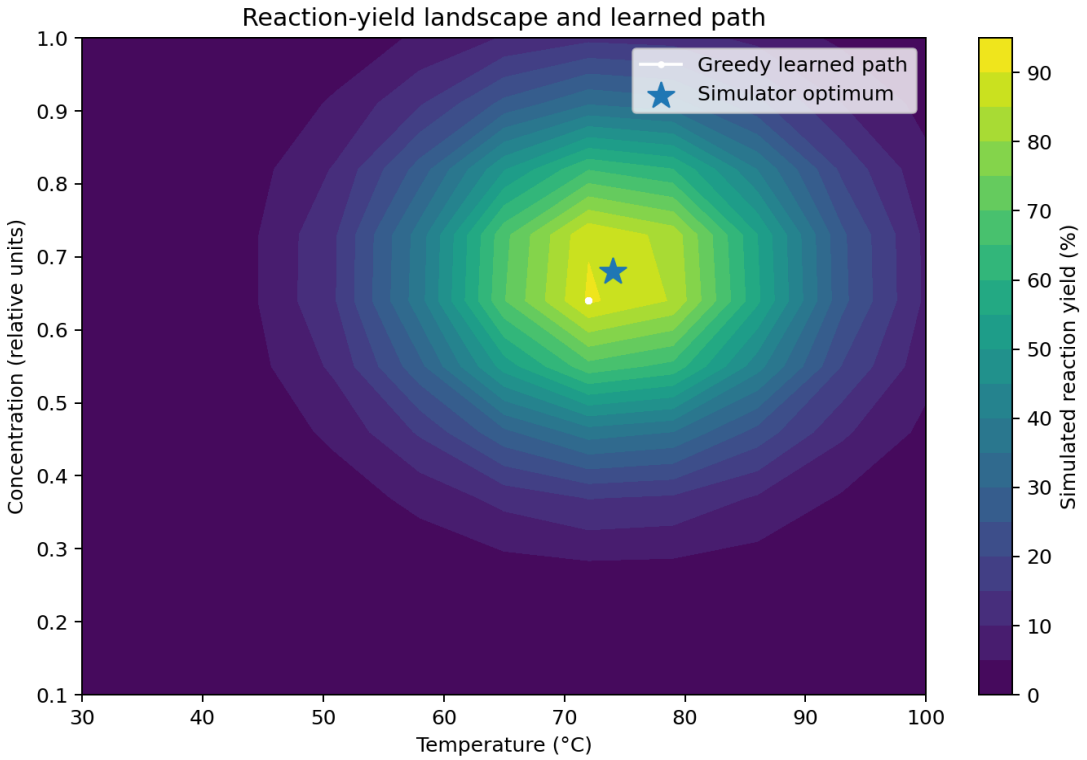


Figure 2. Synthetic yield landscape and the greedy policy's path during evaluation. The star marks the simulator's known optimum; it is shown for evaluation only and is not supplied to the agent.

4. Interpretation and limitations

The learning curve indicates whether the policy improves its experimental choices relative to random exploration. The synthetic response surface has a designed optimum near 74 °C and concentration 0.68. Because this is a toy simulator, success demonstrates the RL workflow rather than real chemical discovery. The model ignores reaction kinetics, safety constraints, equipment limits, chemical identity, cost, and uncertainty in real laboratory measurements. A stronger follow-up would use real experimental data, uncertainty-aware Bayesian optimization, safety constraints, and repeated runs across multiple random seeds.

5. Ethics and responsible use

No real chemicals are handled and no laboratory procedure is being recommended. Real autonomous experimentation requires expert supervision, validated models, equipment safeguards, chemical safety review, and careful handling of experimental data. The simulator's yield values must not be interpreted as predictions for an actual reaction.

6. Conclusion

This project demonstrates how an RL agent can sequentially adjust experimental conditions and learn a policy that seeks higher reaction yield. Q-learning provides an interpretable baseline for a small discrete laboratory simulator.

Reproducibility: Python 3, NumPy and Matplotlib. Run `python self_driving_lab.py`. The supplied training_results.csv and figures were generated with seed 42.